Images: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Hands-On Gameboy-Entwicklung

DI Dr. Tristan Repolusk

Grazer Linuxtage 2026
Graz, 10.04.2026

LibreOffice

00 Setup

15:00–15:10
Link zu Repo & Folien
Installation benötigter Software

02 Drei Regeln

Wichtige Regeln!
Sonst evtl. Hardwareschäden

01 Intro

15:10–15:50
Motivation: Warum Gameboy?
Hardware und Komponenten
Gameboy-Assembly
Entwicklungsumgebung
Pause

03 Hands-On

16:00–ca. 17:15
Pause
ca. 17:30–18:50

Let’s go!

00. Setup

Kursmaterialien

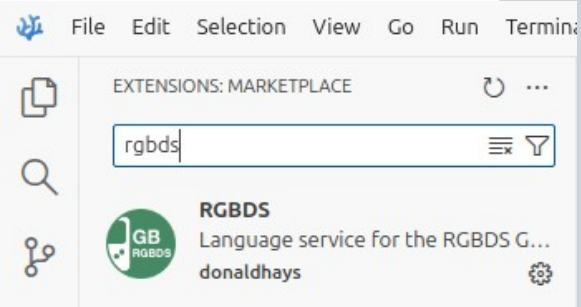
- Repository
  https://github.com/meteoritenhagel/glt26-gameboy-workshop/

- Slides: gameboy-hands-on-DE.pdf

Installation

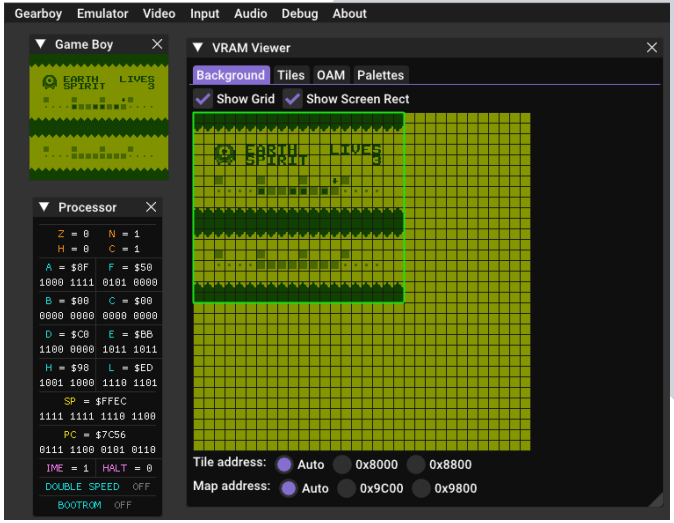
RGBDS (MIT License): Assembler/Linker

- Installation (https://github.com/gbdev/rgbds)
  - Download https://github.com/gbdev/rgbds/releases/latest/download/rgbds-linux-x86_64.tar.xz
  - Extrahieren und `install.sh` ausführen (braucht z. B. `binutils`)
- Testen: Kommando > `rgbasm -V`
  - Sollte zurückgeben: `rgbasm v1.0.1`



Installation

Gearboy (GPL-3.0 License): Emulator/Debugger



- Installation (https://github.com/drhelius/Gearboy/)
  - Snap:
    `snap install gearboy`
  - Aura:
    siehe https://aur.archlinux.org/packages/gearboy
  - Direkter Download:
    https://github.com/drhelius/Gearboy/releases/tag/3.7.5
- Alternativ (free-to-use, aber **nicht quelloffen**): https://emulicious.net/

Installation

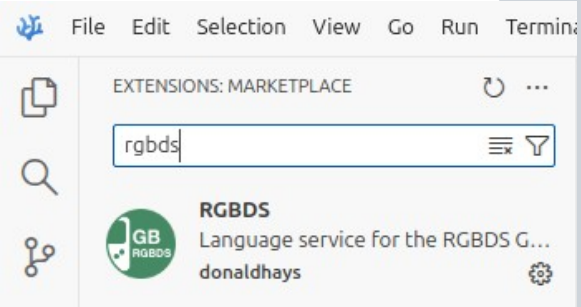
make (GPL-3.0 License): Build-Management

- Installation (https://www.gnu.org/software/make/)
  - Apt:
    `sudo apt install make`
  - Arch: siehe https://archlinux.org/packages/core/x86_64/make/
  - Fedora: siehe https://packages.fedoraproject.org/pkgs/make/make/index.html
- Testen: Kommando `make -v`
  - Sollte zurückgeben: z. B. `GNU Make 4.3`

Installation

Text Editor, z. B. VSCodium (MIT License)

- Installation (https://vscodium.com/)
  - Snap:
    `snap install codium --classic`
  - Aura:
    `sudo aura -A vscodium-bin`
  - Flatpak:
    `flatpak install flathub com.vscodium.codium`
- RGBDS-Extension (MIT License):
  https://github.com/DonaldHays/rgbds-vscode
  - File > Preferences > Extensions



01. Intro

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Warum Gameboy?

- Simple Architektur
- Gut dokumentiert
- Mit vielen Geräten abspielbar
- Hardware leicht erhältlich
- Nostalgie

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Game Boy (1986)

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Super Game Boy (1994)

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Game Boy Pocket (1996)

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Game Boy Color (1998)

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Game Boy Advance (2001)

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Game Boy Player (2003)

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Game Boy Advance SP (2003)

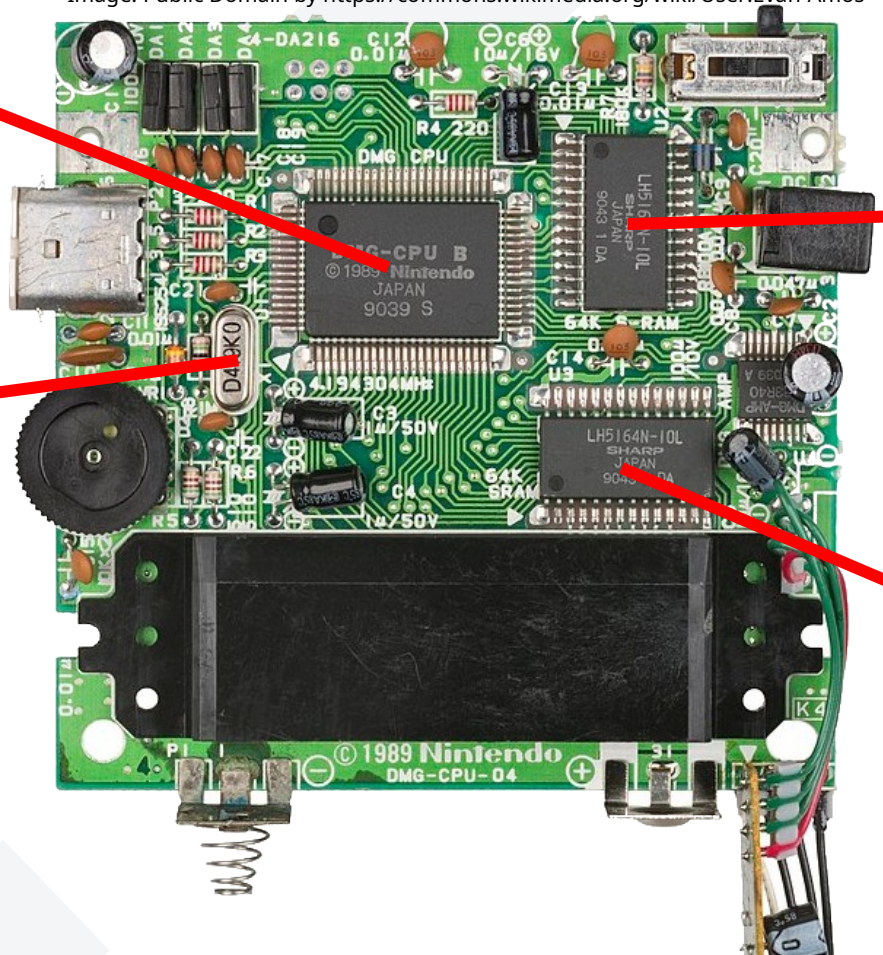
Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

Hardware

System-on-a-Chip
Sharp LR35902
(CPU, PPU, APU)

4.19 MHz crystal
oscillator

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Evan-Amos

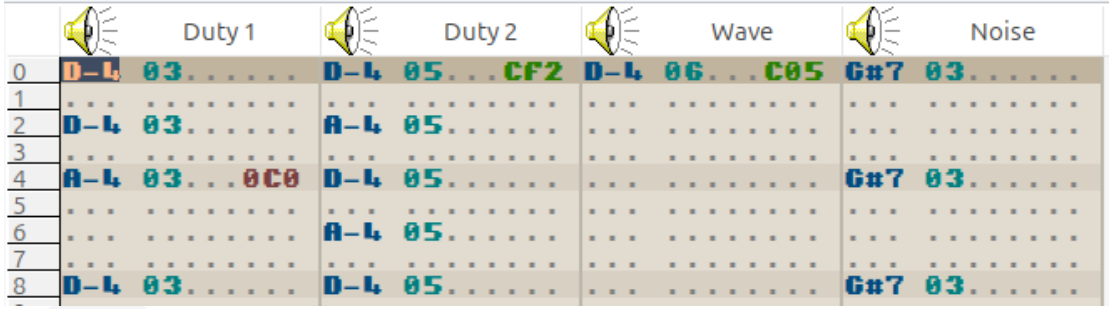


8 KB Video RAM

8 KB Working RAM

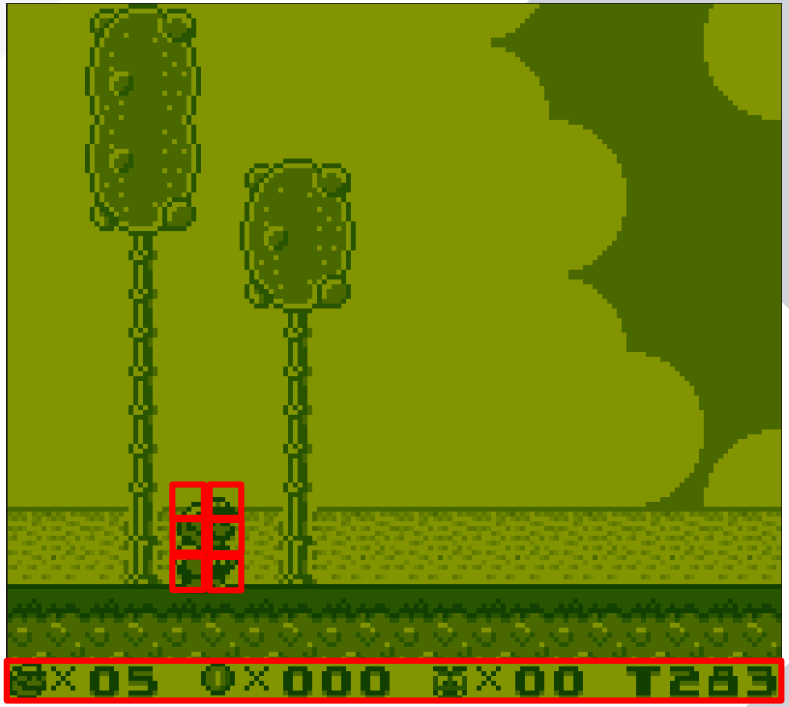
APU (Audio Processing Unit)

- Vier verschiedene Channel
  (Square 1, Square 2, Wave, Noise)
- Kann direkt mit Befehlen von der CPU angesteuert werden
  (Memory Mapped I/O)
- Wir nutzen hier die Library hUGEDriver (Public Domain)
  https://github.com/SuperDisk/hUGEDriver



PPU (Picture Processing Unit)

- Drei verschiedene Layer:
  - Background Layer (8x8-Tiles): Umgebung
  - Window Layer (8x8-Tiles, stark eingeschränkt): Statusleiste
  - Object Layer (8x8 bzw. 8x16 Pixel, pixelweise Position): Projektile
- Wir beschränken uns auf den **Background Layer**



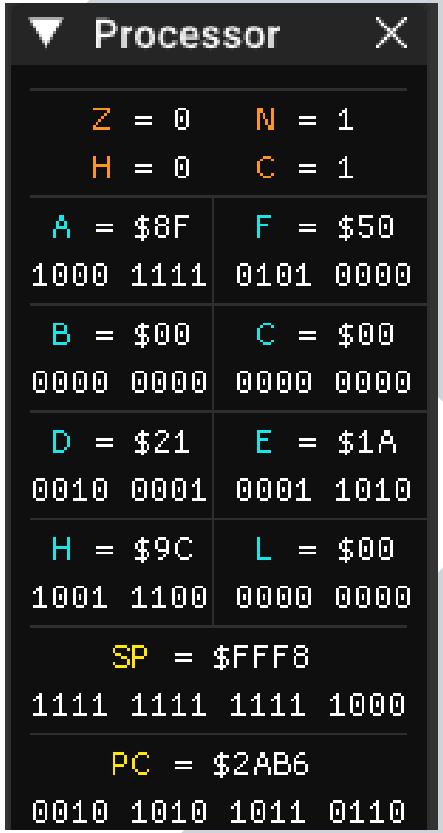
Quelle: *Super Mario Land 2. 6 Golden Coins* [Nintendo Game Boy]. (1992). Nintendo.

PPU (Picture Processing Unit)

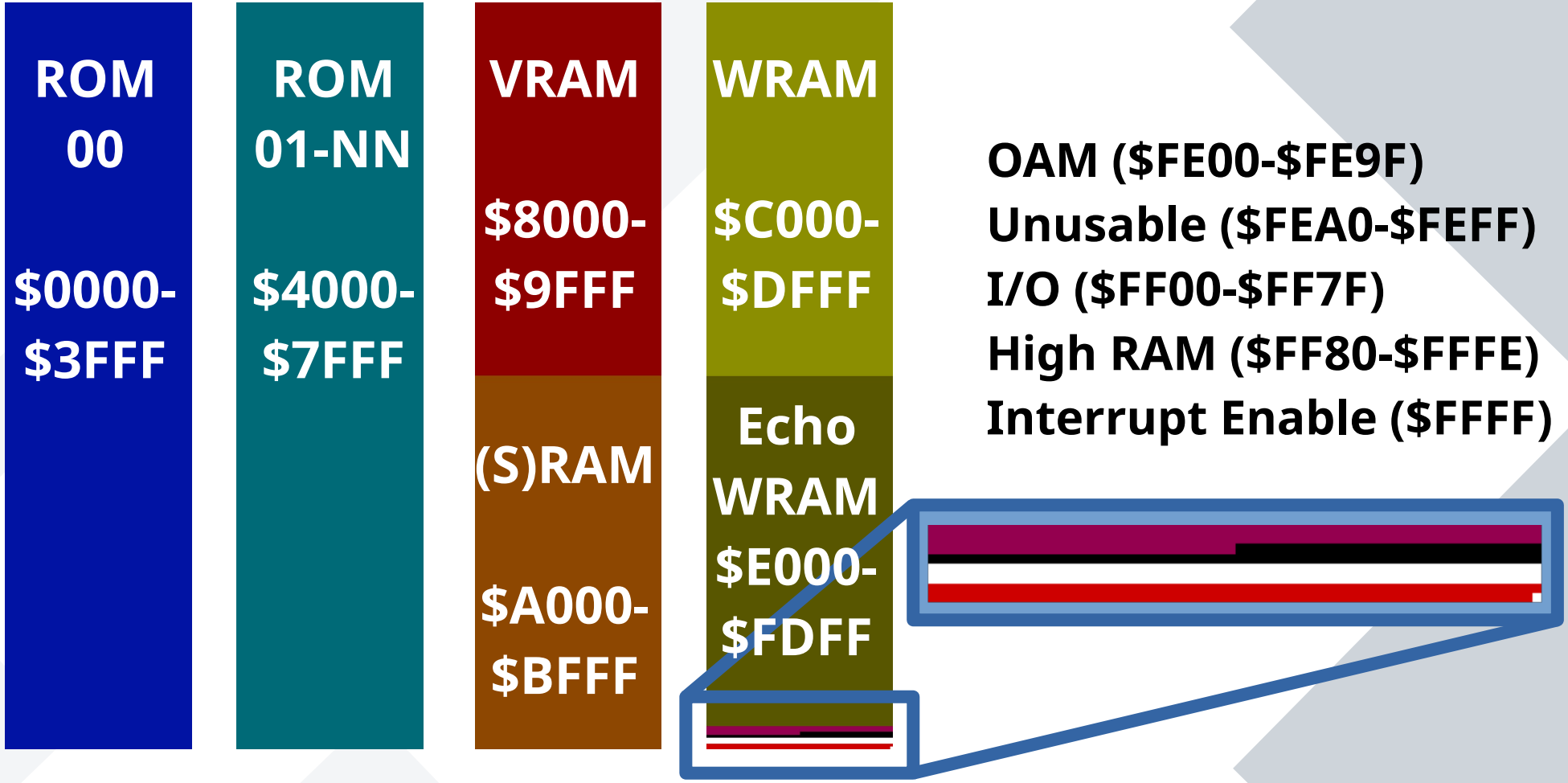
- 8 KB VRAM
- Bilder werden im 2bpp-Format gespeichert
  - 2 Bits pro Pixel → Werte 0, 1, 2 und 3
  - 1 Tile = 8x8 Pixel → 128 Bits = 16 Bytes

CPU (Central Processing Unit)

- Sharp SM83
- 16-Bit Addressraum
- Elemente vom Intel 8080 und Zilog Z80
- Register: af (Accumulator + Flags), bc, de, hl
- Flags: **z**, n, h, **c**
  (Zero, Subtraction, Half-Carry, Carry)
- Stack Pointer (SP) und Program Counter (PC)
- Memory Mapped I/O



Memory Map


ROM 00
$0000-$3FFF
ROM 01-NN
$4000-$7FFF
VRAM
$8000-$9FFF
(S)RAM
$A000-$BFFF
WRAM
$C000-$DFFF
Echo WRAM
$E000-$FDFF
OAM ($FE00-$FE9F)
Unusable ($FEA0-$FEFF)
I/O ($FF00-$FF7F)
High RAM ($FF80-$FFFE)
Interrupt Enable ($FFFF)

Memory Editor

File Edit Selection Bookmarks Watches Search

BANKS: ROM1 $01 RAM $00 WRAM1 $01 VRAM $00

ROM0 ROM1 VRAM RAM WRAM OAM IO HIRAM

```
ADDR    00 01 02 03  04 05 06 07  08 09 0A 0B  0C 0D 0E 0F    ASCII
0000:   23 2F 30 39  00 34 28 25  00 32 28 39  34 28 2D 01    #/09.4(%.2(94(-.
0010:   FF 39 2F 35  FF 39 2F 35  00 37 2F 2E  01 FF 00 00    .9/5.9/5.7/.....
0020:   00 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00    ................
0030:   00 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00    ................
0040:   C3 E6 00 AF  EA 64 C0 21  45 18 CD 50  09 3E 01 EA    .....d.!E..P.>..
0050:   64 C0 CD AE  2A AF EA 40  FF 11 50 01  21 00 90 01    d...*..@..P.!...
0060:   00 08 CD 76  2A 11 BF 1D  21 00 98 01  00 04 CD 76    ...v*...!......v
0070:   2A 3E 81 EA  40 FF 3E FF  EA 47 FF CD  3E 2B 1E 1E    *>..@.>..G..>+..
```

0000 GoTo 00 Find Next REGION: 0000-3FFF SELECTION: 0048

Dec: 69 (Uint8)

Hex: 45

Bin: 0100 0101

Options

Assembly

- Maschinensprache:
  - Befehl = Opcode + Daten,
  - Beispiel: GB-Z80, lade Wert 255 in Register A:
    `00111110 11111111` bzw. `0x3E 0xFF`
- Assemblysprachen:
  - 1:1-Beziehung zwischen Assemblycode und Maschinencode
  - Befehl = Mnemonic + Daten
  - Beispiel: RGBASM, lade Wert 255 in Register A:
    `LD A, $FF`

Assembly

- Höhere Programmiersprachen (z. B. C, C++, Python, ...):
  - Höherer Abstraktionsgrad, leichter verständlich
  - Beispiel (C):
    Kopiere die ersten `n` Bytes von `source` nach `destination`

```
void memcopy(void *destination, const void *source, size_t n) {
    uint8_t *dst = destination;
    const uint8_t *src = source;
    for (size_t i = 0; i < n; i++) {
        dst[i] = src[i];
    }
}
```

C

```c
void memcopy(void *destination, const void *source, size_t n) {
    uint8_t *dst = destination;
    const uint8_t *src = source;
    for (size_t i = 0; i < n; i++) {
        dst[i] = src[i];
    }
}
```

x86-64 ASM

```asm
; rsi: source
; rdi: destination
; rdx: n
memcopy:
    test rdx, rdx
    je .L1
    mov eax, 0
.L3:
    movzx ecx, BYTE PTR [rsi+rax]
    mov BYTE PTR [rdi+rax], cl
    add rax, 1
    cmp rdx, rax
    jne .L3
.L1:
    ret
```

GB-Z80 RGBASM

```asm
; de: source
; hl: destination
; bc: n
; only call with bc > 0
; destroys a
memcopy::
    ld a, [de]
    ld [hli], a
    inc de
    dec bc
    ld a, b
    or a, c
    jr nz, memcopy
    ret
```

Wichtigste Befehle

- Ladeinstruktionen:
  - `ld a, $64`: a ← $64
  - `ld a, b`: a ← b (jedes andere Register möglich)
  - `ld [$ABCD], a`: Lade a in das Byte mit Adresse $ABCD
  - `ld a, [$ABCD]`: Lade das Byte mit Adresse $ABCD nach a.
  - …
- Logische Operatoren immer mit dem Accumulator **a**
  - `add a, b` (bzw. `add b`): a ← a + b
  - statt **b** auch möglich: **a**, **c**, **d**, **e**, **h**, **l**, **[hl]**
  - statt **and** andere Operationen: **or** oder **xor**

Wichtigste Befehle

- Kein If/While/For
- Dafür Sprunganweisungen:
  - Jump: `jp label`
  - Relative jump: `jr label`
  - Call subroutine: `call label`
  - Return from subroutine: `ret`, `reti`
- Bedingte Sprünge (Zero Flag, Carry Flag):
  - Jump if zero: `jp z, label`; jump if not zero: `jp nz, label`
  - Jump if carry: `jp c, label`; jump if not carry: `jp nc, label`
  - Äquivalent für `jr`, `call` und `ret`

Image: Public Domain by https://commons.wikimedia.org/wiki/User:Ocdp

Kontrollstrukturen

Wir simulieren If-Statements:

```
CheckEqual::
    ld a, [wVariable]
    cp 5                        ; if [wVariable] == 5
    call z, VariableEqualFive ; call subroutine VariableEqualFive
    ret

CheckLess::
    ld a, [wVariable]
    cp 5                       ; if [wVariable] < 5
    call c, VariableLessFive ; call subroutine VariableLessFive
    ret

CheckGreaterOrEqual::
    ld a, [wVariable]
    cp 5                                ; if [wVariable] >= 5
    call nc, VariableGreaterOrEqualFive ; call subroutine GreaterOrEqualFive
    ret
```

Kontrollstrukturen

Wir simulieren For-Statements:

```
ForLoop::
    ld a, 10        ; iterate 10 times
.loop
    ; do something
    dec a
    jr nz, .loop
    ret
```

Kontrollstrukturen

Wir simulieren Case-Statements:

```
CaseStatement::
    ld a, [wVariable]
    cp 0                    ; in case [wVariable] == 0
    jr nz, .notZero
    call VariableIsZero     ; call subroutine VariableIsZero
    jr .done
.notZero
    cp 1                    ; in case [wVariable] == 1
    jr nz, .notOne
    call VariableIsOne      ; call subroutine VariableIsOne
    jr .done
.notOne
    cp 2                    ; in case [wVariable] == 2
    jr nz, .done
    call VariableIsTwo      ; call subroutine VariableIsTwo
.done
    ret  ; or other exit, like jp, jr, ...

; ...
```

Kurze Pause

02. Drei Regeln

Drei Regeln

1) Display ausschalten vor VRAM-Zugriffen

2) Display nur während VBlank ausschalten! (Hardwareschaden)

3) Endlosschleifen statt Runaway-Code

03. Hands-On

Empty Project

git checkout 00-empty-project

- main.asm
  - ROM Header
  - Entry Point
  - Busy Waiting
- utils.asm
  - Memcopy
  - UpdateKeys

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/00-empty-project...01-title-screen-display

Ü1: Grafiken anzeigen

Voraussetzung: git checkout 00-empty-project

Ziel: Den Title Screen anzeigen

- title.png von raw_resources nach project/res kopieren
- utils.asm: Schreibe die Funktion WaitVBlank
- Neues File state_title.asm anlegen
  - Schreibe InitStateTitle
  - WaitVBlank + LCD ausschalten
  - Tiles ($9000) und Tilemap ($9800) einbinden
  - Tiles kopieren, dann Tilemap kopieren
  - LCD einschalten und Palette initialisieren
- main.asm: InitStateTitle aufrufen

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/01-title-screen-display...02-title-screen-change-palette

Ü2: Text blinken lassen

Voraussetzung: git checkout 01-title-screen-display

Ziel: “Press Start” zum Blinken bringen

- utils.asm: Schreibe die Funktion WaitMultipleVBlank, welche eine bestimmte Anzahl von VBlanks abwartet
- state_title.asm: Wechsle zwischen Paletten nach einer halben Sekunde, damit der Schriftzug “Press Start” blinkt

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/02-title-screen-change-palette...03-title-screen-react-on-player-input

Ü3: Auf Eingabe reagieren

Voraussetzung: git checkout 02-title-screen-change-palette

Ziel: Den Titelbildschirm verlassen, wenn eine Taste gedrückt wird

> state_title.asm
> - Neue Funktion WaitMultipleVBlankPressStart, welche jedes Frame auf Input überprüft
> - Wenn Input erkannt wurde, verlassen wir den Title-Screen durch `ret`

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/03-title-screen-react-on-player-input...04-title-screen-add-music

Ü4: Musik hinzufügen

Voraussetzung:
git checkout 03-title-screen-react-on-player-input

Ziel: Ein Musikstück zum Klingen bringen

- Verschieben von raw_resources in das Projekt:
  - hUGEDriver/hUGE.inc → include
  - hUGEDriver/hUGE_note_table.inc → include
  - hUGEDriver/hUGEDriver.asm → src
  - huge_music/music_title.asm → src
- variables.asm: wUpdateSound
- start_title.asm: Laden des Musikstücks
- main.asm: Initialisieren + VBlank Interrupt Handler
  - **!!!**

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/04-title-screen-add-music...05-switch-between-game-states

Ü5: Mehrere Game-States

Voraussetzung: git checkout 04-title-screen-add-music

Ziel: Den Titelbildschirm verlassen, wenn eine Taste gedrückt wird

- constants.inc: Füge zwei Konstanten für die beiden Game-States Title und Game hinzu
- variables.asm: wNextState
- Neues File: state_game.asm
  - Funktion InitStateGame
- main.asm: Statt .loop machen wir eine Entscheidung (wNextState) → entweder nach InitStateTitle oder InitStateGame
- title.asm: Vorm Return laden wir noch den nächsten geplanten State, also Game, nach wNextState

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/05-switch-between-game-states...06-gameplay-display

Ü6: Grafiken vom Game-State

Voraussetzung: git checkout 05-switch-between-game-states

Ziel: Die Grafik für die Spielmechanik anzeigen

- game.png von raw_resources nach res verschieben
- utils.asm: Neue Funktion StopSounds, welche alle vier Soundchannels stummschaltet
- state_game.asm:
  - Soundchannel stummschalten
  - Grafiken einbinden und anzeigen (vgl. state_title.asm)

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/06-gameplay-display...07-draw-text

Ü6: Text anzeigen

Voraussetzung: git checkout 06-gameplay-display

Ziel: Die Grafik für die Spielmechanik anzeigen

- Verschieben von raw_resources nach project:
  - images/charmap.inc → include
  - images/font.png → res
- utils.asm: Neue Funktion DrawText, welche Text anzeigt
- state_game.asm:
  - Textdaten anlegen
  - Texttiles laden ($8800)
  - Richtige Startposition für Text finden
  - DrawText aufrufen

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/07-draw-text...08-gameplay-metronome

Ü7: Metronom (Teil 1)

Voraussetzung: git checkout 07-draw-text

Ziel: Schlaggeräusch abspielen

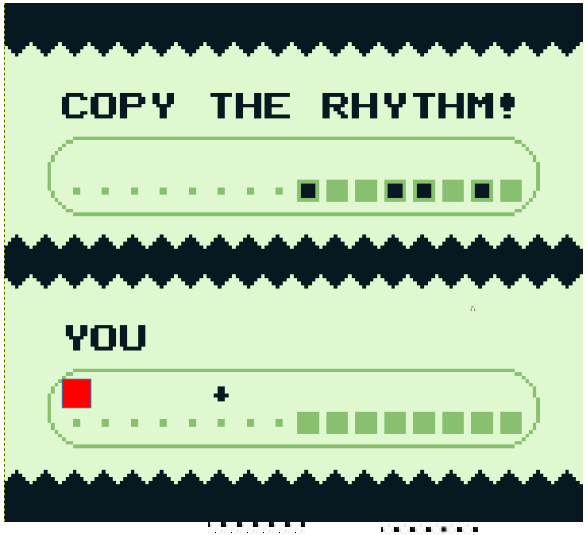
- Kopieren huge_music/music_weakbeat.asm → src
- utils.asm:
  - Neue Funktion PlayWeakBeat
  - Soll schwachen Schlag abspielen (in music_weakbeat.asm)
  - Push alle Register
  - Laden des Sounds (siehe state_title.asm)
  - Pop alle Register
- Gerne testen!

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/07-draw-text...08-gameplay-metronome

Ü7: Metronom (Teil 2)

Ziel: Spielmechanik Metronom implementieren

- Startposition finden (rot markiert)
- Tile Number von Hintergrund/Pfeil finden?
- state_game.asm:
  - Lade Startposition nach hl
  - Funktion PlayMetronome schreiben:
    - PlayWeakBeat
    - Wechsle Grafik von Hintergrund auf Pfeil
    - Warte ein bisschen
    - Wechsle Grafik zurück auf leeres Feld
    - Insgesamt für alle acht Felder!



https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/08-gameplay-metronome...09-gameplay-react-on-player-input

Ü8: Eingabe im Spiel (Teil 1)

Voraussetzung: git checkout 08-gameplay-metronome

Ziel: Lautes Schlaggeräusch abspielen

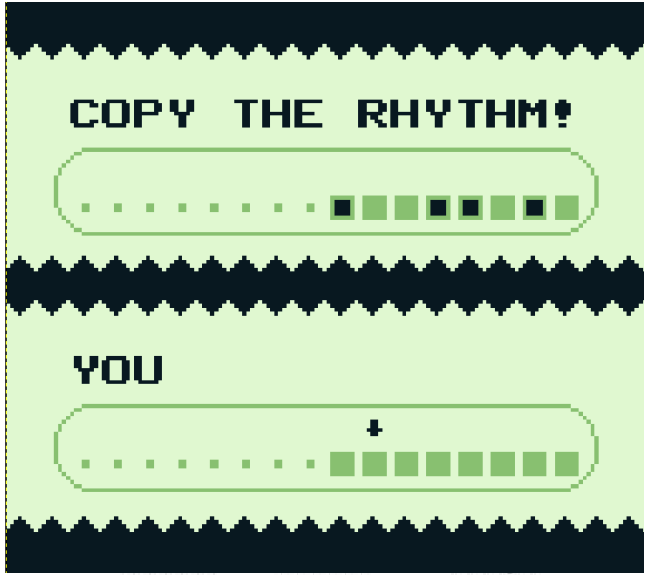
- huge_music/music_strongbeat.asm → ?
- utils.asm: PlayStrongBeat schreiben

https://github.com/meteoritenhagel/glt26-gameboy-workshop/compare/08-gameplay-metronome...09-gameplay-react-on-player-input

Ü8: Eingabe im Spiel (Teil 2)

Ziel: Spielereingabe → Box wird schwarz

- Tile Number von leerer Box bzw. voller Box finden
- state_game.asm:
  - PlayerInput als Kopie von PlayMetronome anlegen
  - Statt WaitMultipleVBlank, neue Funktion WaitMultipleVBlankPlayerInput schreiben (vgl. state_title.asm)
  - Auf Playerinput prüfen:
    - falls ja: PlayStrongBeat und ändere Box auf voll ($20 auf die Adresse addieren geht in nächste Zeile)
    - falls nein: Mach normal weiter



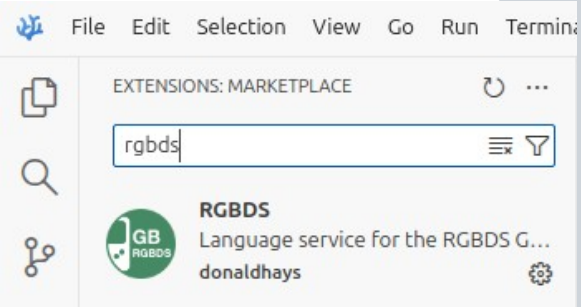
Weitere Übungen

- **10-gameplay-clear-boxes:**
  Mehrere Gameplayrunden mit Zurücksetzen der Boxen
- **11-gameplay-given-beat-pattern:**
  Pseudozufallszahlen als Startwert für den zu wiederholenden Beat
- **12-gameplay-store-player-input:**
  Der Player-Input wird in eine Variable geschrieben
- **13-gameplay-compare-given-and-player-inputs:**
  Wir vergleichen den gegebenen Beat mit dem Player-Input
- **14-gameplay-change-tempo-each-round:**
  Bei jeder erfolgreichen Runde wird das Tempo erhöht
- **15-gameplay-add-success-text:**
  Ist das Spiel durchgespielt, erscheint ein Text “YOU WON!”
- **16-nice-screen-transitions:**
  Jetzt noch ein bisschen die Übergänge zwischen Screens aufpolieren

Appendix

Nützliche Links

- Gameboy-Opcode-Listen: https://meganesu.github.io/generate-gb-opcodes/ (Machine Cycles)
- Gameboy-CPU-Befehle: https://rgbds.gbdev.io/docs/master/gbz80.7
- Allgemeines Tutorial: https://gbdev.io/gb-asm-tutorial/
- Gameboy-Pandocs: https://gbdev.io/pandocs/
- Development-Wiki: https://gbdev.gg8.se/wiki/articles/Main_Page
- Sprite-Manipulation: https://gbdev.gg8.se/wiki/articles/OAM_DMA_tutorial
- Assets:
  - https://opengameart.org/
  - https://itch.io/game-assets



Schluss